Game Development Masterclass

Full-Stack Game Engineering Curriculum

Deep-Dive into Unity (C#) & Unreal Engine 5 (C++) Engineering

1 Mission Statement

Designed for 12th-pass students and graduates, this 52-week masterclass focuses on the **Engineering "Why"** behind game development. We don't just teach you to follow tutorials; we teach you the low-level logic, memory management, and architectural patterns used by professional AAA studios.

2 Admissions & Enrollment

Category	Detail
Eligibility	12th Pass (HSC), Engineering Graduates, or IT Career-Pivoters.
Skill Level	**Absolute Beginner.** We start from "What is code?" and end at "How to optimize memory."
Trial	**1-Week Free Trial** to experience the intensity and quality of the lectures.
Fees	₹24,000 Total (Subscription Model: **₹2,000/month**).

3 Phase 1: Foundations of Programming (Weeks 1–12)

3.1 Basic Syntax & Logic Building

- **Variables & Constants:**
  - *Purpose:* To create "containers" for game information like Health, Ammo, or Player Name.
  - *The "Why":* Computers need a way to track changing data. Variables allow us to reserve space in RAM, retrieve that data using a name, and modify it as the game state changes (e.g., losing health).
- **Data Types (int, float, bool, string):**
  - *Purpose:* Identifying the "nature" of the data so the CPU knows how much memory to allocate.
  - *The "Why":* Efficiency and Precision. Using an `int` is faster for discrete counts (inventory items), whereas a `float` is mandatory for smooth decimal-based calculations like character velocity or world position.
- **Arithmetic & Logical Operators:**
  - *Purpose:* Performing math (adding score) and checking conditions (is the player dead?).
  - *The "Why":* Gameplay is fundamentally a set of mathematical outcomes. Logical operators allow the game to decide "if" a player can jump or "if" an enemy should shoot based on distance math.
- **Control Flow (If-Else, Switch Statements):**
  - *Purpose:* Creating "Branches" in the game code.
  - *The "Why":* Interaction requires choice. Control flow allows the computer to skip or run specific code blocks based on real-time triggers, moving the game from a static script to an interactive experience.
- **Loops (For, While, Foreach):**
  - *Purpose:* Repeating logic across collections (e.g., checking health for all 50 enemies).
  - *The "Why":* Game worlds are massive. Loops allow us to process thousands of objects or data points per frame without writing thousands of lines of code.

3.2 Unity Engine & 2D Architecture

- **Transform & Vector3:** The mathematical foundation of positioning items in 3D/2D space.
- **The Update Loop:** Understanding how the game engine "thinks" and renders every single frame.
- **Prefab Workflow:** Creating reusable game templates to save development time and maintain consistency.
- **Project 1 Deliverable:** An **Endless Runner** published to Itch.io.

4 Phase 2: Advanced System Design & AI (Weeks 13–26)

4.1 Object-Oriented Programming (OOP) Deep-Dive

- **Classes & Objects:** Creating blueprints for complex game actors.
- **Inheritance:**
  - *Purpose:* Sharing common logic (e.g., "Health" or "Damage") across many different enemy types.
  - *The "Why":* Maintainability. It allows us to update the code for "Damage" in one place and have it apply to Archers, Knights, and Bosses instantly.
- **Interfaces & Polymorphism:**

- *Purpose:* Allowing the player to interact with many different types of objects using one command.
- *The "Why":* Decoupling. A "Bullet" can hit a "Prop" or an "Enemy"; if both use an `IDamageable` interface, the bullet doesn't need to know exactly what it hit to deal damage.

4.2 Engine Optimization & Patterns

- **Object Pooling:**
  - *Purpose:* Reusing bullet or particle objects instead of creating and destroying them.
  - *The "Why":* Performance. Creating/Destroying objects triggers "Garbage Collection," which causes visible lag spikes. Pooling keeps the game smooth.
- **The Observer Pattern:** Using "Events" to let the UI know the health changed without the health script needing to know the UI exists.
- **NavMesh & AI:** Using Pathfinding algorithms to let enemies navigate complex environments intelligently.
- **Project 2 Deliverable:** A **3D Action Game** with intelligent enemy AI.

5 Phase 3: Unreal Engine 5 & C++ Engineering (Weeks 27–40)

5.1 Low-Level C++ Foundations

- **Pointers & Memory Addresses:**
  - *Purpose:* Directly interacting with the computer's memory to find where data is stored.
  - *The "Why":* In AAA development, pointers allow different game systems to share massive datasets (like a 3D model) instantly without the CPU cost of copying the data.
- **Stack vs. Heap Memory:**
  - *Purpose:* Understanding temporary memory (Stack) vs. persistent memory (Heap).
  - *The "Why":* Proper memory management prevents "Memory Leaks." Controlling exactly when data is deleted is why C++ games run faster than those in managed languages.
- **Header (.h) vs. Implementation (.cpp):**
  - *Purpose:* Separating the "Declaration" (what it is) from the "Definition" (how it works).
  - *The "Why":* Faster compile times. It allows the compiler to only process changed code, which is critical for projects with millions of lines of code.

5.2 The UE5 Gameplay Framework

- **Macros (UPROPERTY, UFUNCTION):** The bridge that exposes high-performance C++ code to the Unreal Editor.
- **Actors & Components:** Building performance-critical gameplay features using the industry-standard AAA framework.
- **Nanite & Lumen Implementation:** Designing environments that utilize real-time dynamic lighting and virtualized geometry.
- **Project 3 Deliverable:** A **C++ Based Combat Mechanic Showcase** in Unreal Engine 5.

6 Phase 4: Networking & Professional Career (Weeks 41–52)

6.1 Online Systems & Databases

- **Client-Server Replication:** Syncing health, movement, and animations across 50+ players over the internet.
- **REST APIs & Databases:**
  - *Purpose:* Saving player accounts, inventories, and global leaderboards to a remote server.
  - *The "Why":* To ensure player data is secure and consistent across different devices, preventing cheating and data loss.

6.2 Capstone & Placement Preparation

- **Technical Profiling:** Using the GPU/CPU profiler to identify and fix performance bottlenecks or "Lag Spikes."
- **Mock Technical Interviews:** Practicing whiteboard coding, C++/C# theory, and algorithm solving.
- **The Capstone:** A 12-week production cycle to build a professional-grade Vertical Slice for your portfolio.
- **Final Event: Jainish.Space Demo Day** — Presenting your final game to industry partners and hiring studios.

Register for the April Cohort at learn.jainish.space

Master the engineering. Create the world.